

Progetto di Calcolo Numerico

Tommaso Vaccari

Aprile 2026

Indice

1	Analisi circuitale: modello matematico	2
1.1	Leggi di Kirchhoff alle correnti	2
1.2	Legge di Ohm per le correnti di lato	2
1.3	Legge di Kirchhoff alla tensione nella maglia di ingresso	3
1.4	Incognite ed equazioni	3
2	Analisi circuitale con metodi diretti	3
2.1	Costruzione della matrice delle ammettenze e del termine noto	3
2.2	Esistenza e unicità della fattorizzazione LU	5
2.3	Calcolo della fattorizzazione LU con <code>lu</code>	6
2.4	Risoluzione con la fattorizzazione LU	7
2.5	Risoluzione con l'operatore <code>\</code>	7
2.6	Confronto tra le due soluzioni	7
3	Analisi circuitale con metodi iterativi	8
3.1	Verifica che la matrice è simmetrica definita positiva	8
3.2	Metodo di Jacobi	8
3.3	Raggio spettrale della matrice di iterazione di Jacobi	9
3.4	Metodo del gradiente dinamico	9
3.5	Metodo del gradiente preconditionato	10
3.6	Metodo del gradiente coniugato	10
3.7	Grafico del residuo normalizzato	11
3.8	Numeri di condizionamento e commento finale	12

1 Analisi circuitale: modello matematico

In questa prima parte ho costruito il modello del circuito partendo dalle leggi di Kirchhoff e dalla legge di Ohm. Ho adottato la convenzione indicata nel testo: una corrente che attraversa un nodo è positiva se uscente dal nodo stesso.

1.1 Leggi di Kirchhoff alle correnti

Indicando con i_{in} la corrente di ingresso e con $i_1, \dots, i_{15}$ le correnti interne del circuito, le equazioni di Kirchhoff ai nodi sono:

$$\text{Nodo 1: } i_3 + i_1 - i_{in} = 0$$

$$\text{Nodo 2: } i_4 + i_2 - i_1 = 0$$

$$\text{Nodo 3: } i_5 - i_2 = 0$$

$$\text{Nodo 4: } i_8 + i_6 - i_3 = 0$$

$$\text{Nodo 5: } i_9 + i_7 - i_6 - i_4 = 0$$

$$\text{Nodo 6: } i_{10} - i_7 - i_5 = 0$$

$$\text{Nodo 7: } i_{13} + i_{11} - i_8 = 0$$

$$\text{Nodo 8: } i_{14} + i_{12} - i_{11} - i_9 = 0$$

$$\text{Nodo 9: } i_{15} - i_{12} - i_{10} = 0$$

Queste sono le 9 equazioni ai nodi della rete.

1.2 Legge di Ohm per le correnti di lato

Per ogni ramo ho usato la relazione

$$G_k = \frac{1}{R_k}, \quad i_k = G_k(v_x - v_y),$$

dove $v_x - v_y$ è la differenza di potenziale ai capi del resistore orientata nel verso della corrente scelta.

Nel mio modello si ottiene quindi:

$$\begin{aligned} i_1 &= G_1(v_1 - v_2), & i_2 &= G_1(v_2 - v_3), \\ i_3 &= G_2(v_1 - v_4), & i_4 &= G_2(v_2 - v_5), \\ i_5 &= G_2(v_3 - v_6), & i_6 &= G_3(v_4 - v_5), \\ i_7 &= G_3(v_5 - v_6), & i_8 &= G_4(v_4 - v_7), \\ i_9 &= G_4(v_5 - v_8), & i_{10} &= G_4(v_6 - v_9), \\ i_{11} &= G_5(v_7 - v_8), & i_{12} &= G_5(v_8 - v_9), \\ i_{13} &= G_6 v_7, & i_{14} &= G_6 v_8, \\ i_{15} &= G_6 v_9. \end{aligned}$$

Per la corrente di ingresso, dalla maglia con il generatore e il resistore R_{in} si ricava

$$i_{in} = G_{in}(V_{in} - v_1), \quad G_{in} = \frac{1}{R_{in}}.$$

1.3 Legge di Kirchhoff alla tensione nella maglia di ingresso

Considerando la maglia formata dal generatore V_{in} e dai resistori R_{in} , R_2 , R_4 e R_6 , ho scritto:

$$-i_{in}R_{in} + V_{in} - i_3R_2 - i_8R_4 - i_{13}R_6 = 0.$$

Questa relazione è coerente con il fatto che la caduta totale di tensione sulla catena di resistori deve eguagliare la tensione imposta dal generatore.

1.4 Incognite ed equazioni

Le incognite del problema sono:

- 15 correnti interne $i_1, \dots, i_{15}$;
- 1 corrente di ingresso i_{in} ;
- 9 tensioni nodali $v_1, \dots, v_9$.

In totale le incognite sono quindi 25.

Le equazioni disponibili sono:

- 9 equazioni di Kirchhoff alle correnti;
- 15 equazioni di Ohm sui lati;
- 1 equazione di Kirchhoff alle tensioni sulla maglia di ingresso.

Anche le equazioni sono quindi 25. Il modello risulta quadrato e il numero di equazioni coincide con il numero di incognite.

2 Analisi circuitale con metodi diretti

In questa parte ho assegnato i valori numerici richiesti dal testo:

$$V_{in} = 15 \text{ V}, \quad R_{in} = 60 \Omega, \quad R_k = \frac{R_{in}}{2k}, \quad k = 1, \dots, 6.$$

Da qui segue che

$$G_{in} = \frac{1}{60}, \quad G_k = \frac{1}{R_k} = 2k G_{in}.$$

2.1 Costruzione della matrice delle ammettenze e del termine noto

Eliminando le correnti di ramo mediante la legge di Ohm e sostituendole nelle KCL, il sistema nodale assume la forma

$$Ax = b, \quad x = [v_1 \ v_2 \ \dots \ v_9]^T.$$

Le nove equazioni ottenute sono:

$$\begin{aligned}
& (G_1 + G_2 + G_{in})v_1 - G_1v_2 - G_2v_4 = G_{in}V_{in}, \\
& -G_1v_1 + (2G_1 + G_2)v_2 - G_1v_3 - G_2v_5 = 0, \\
& -G_1v_2 + (G_1 + G_2)v_3 - G_2v_6 = 0, \\
& -G_2v_1 + (G_2 + G_3 + G_4)v_4 - G_3v_5 - G_4v_7 = 0, \\
& -G_2v_2 - G_3v_4 + (G_2 + 2G_3 + G_4)v_5 - G_3v_6 - G_4v_8 = 0, \\
& -G_2v_3 - G_3v_5 + (G_2 + G_3 + G_4)v_6 - G_4v_9 = 0, \\
& -G_4v_4 + (G_4 + G_5 + G_6)v_7 - G_5v_8 = 0, \\
& -G_4v_5 - G_5v_7 + (G_4 + 2G_5 + G_6)v_8 - G_5v_9 = 0, \\
& -G_4v_6 - G_5v_8 + (G_4 + G_5 + G_6)v_9 = 0.
\end{aligned}$$

Nel codice MATLAB ho definito A e b come segue:

```

Gin = 1 / 60;
Vin = 15;
G1 = 2 * 1 * Gin;
G2 = 2 * 2 * Gin;
G3 = 2 * 3 * Gin;
G4 = 2 * 4 * Gin;
G5 = 2 * 5 * Gin;
G6 = 2 * 6 * Gin;

A = [G1+G2+Gin, -G1, 0, -G2, 0, 0, 0, 0, 0;
     -G1, 2*G1 + G2, -G1, 0, -G2, 0, 0, 0, 0;
     0, -G1, G2+G1, 0, 0, -G2, 0, 0, 0;
     -G2, 0, 0, G4+G3+G2, -G3, 0, -G4, 0, 0;
     0, -G2, 0, -G3, G4 + 2*G3 + G2, -G3, 0, -G4, 0;
     0, 0, -G2, 0, -G3, G4+G3+G2, 0, 0, -G4;
     0, 0, 0, -G4, 0, 0, G6+G5+G4, -G5, 0;
     0, 0, 0, 0, -G4, 0, -G5, G6+ 2*G5+ G4, -G5;
     0, 0, 0, 0, 0, -G4, 0, -G5, G4+ G5 + G6];

b = [Vin * Gin; 0; 0; 0; 0; 0; 0; 0; 0];

```

Con format short e, i valori visualizzati sono:

```

A =
 1.1667e-01 -3.3333e-02 0 -6.6667e-02 0 0 0 0 0
-3.3333e-02 1.3333e-01 -3.3333e-02 0 -6.6667e-02 0 0 0 0
 0 -3.3333e-02 1.0000e-01 0 0 -6.6667e-02 0 0 0
-6.6667e-02 0 0 3.0000e-01 -1.0000e-01 0 -1.3333e-01 0 0
 0 -6.6667e-02 0 -1.0000e-01 4.0000e-01 -1.0000e-01 0 -1.3333e-01 0
 0 0 -6.6667e-02 0 -1.0000e-01 3.0000e-01 0 0 -1.3333e-01
 0 0 0 -1.3333e-01 0 0 5.0000e-01 -1.6667e-01 0
 0 0 0 0 -1.3333e-01 0 -1.6667e-01 6.6667e-01 -1.6667e-01
 0 0 0 0 0 -1.3333e-01 0 -1.6667e-01 5.0000e-01

b =
 2.5000e-01
 0
 0
 0
 0
 0
 0
 0
 0
 0

```

2.2 Esistenza e unicità della fattorizzazione LU

Dal punto di vista teorico, una matrice ammette fattorizzazione LU senza pivoting se e solo se tutti i minori principali di testa sono non nulli. Nel nostro caso conviene però usare prima una proprietà strutturale più forte: la matrice delle ammettenze è simmetrica e definita positiva. Per una matrice simmetrica definita positiva, il criterio di Sylvester garantisce che tutti i minori principali di testa sono strettamente positivi; di conseguenza la fattorizzazione LU senza pivoting esiste ed è unica.

Nel mio script ho quindi verificato prima la proprietà SPD, che è già sufficiente per concludere:

- una condizione necessaria e sufficiente è che tutti i minori principali principali siano non nulli;
- nel caso simmetrico definito positivo tali minori sono addirittura positivi;
- quindi non serve pivoting per garantire l'esistenza della fattorizzazione.

La verifica numerica usata è stata:

```
if ( A == A')
    if( all(eig(A) > 0))
        disp("La matrice e' simmetrica e definita positiva, fattorizzazione" +
            ...
            " LU esiste ed e' unica!")
    end
end
```

Ho comunque controllato anche la dominanza diagonale stretta per righe e per colonne, non perché fosse necessaria, ma per confrontare il caso in esame con un'altra ipotesi classica di applicabilità dell'eliminazione gaussiana senza pivoting:

- una matrice strettamente diagonalmente dominante per righe o per colonne ammette LU senza pivoting;
- qui tale proprietà non vale globalmente, quindi la giustificazione corretta resta la definitezza positiva.

Ho inoltre controllato anche la dominanza diagonale stretta per righe e per colonne:

```
rowDominance = 1;
for i = 1 : n
    rowSum = sum(abs(A(i, :))) - abs(A(i, i));
    if abs(A(i, i)) <= rowSum
        rowDominance = 0;
        disp("Matrice non a dominanza diagonale di riga stretta")
        break;
    end
end

columnDominance = 1;
for j = 1 : n
    columnSum = sum(abs(A(:, j))) - abs(A(j, j));
    if abs(A(j, j)) <= columnSum
```

```

        columnDominance = 0;
        disp("Matrice non a dominanza diagonale di colonna stretta")
        break;
    end
end

```

La dominanza diagonale stretta non vale su tutte le righe e colonne, quindi da sola non basta. Il punto teoricamente rilevante è che A è SPD, e questa ipotesi è già sufficiente per l'esistenza e l'unicità della LU.

Per completezza ho controllato anche i determinanti delle sottomatrici principali:

```

determinants = zeros(n, 1);
for k = 1:n
    subMatrix = A(1:k, 1:k);
    determinants(k) = det(subMatrix);
end

```

I valori ottenuti sono:

```

det(A_1) = 1.166666666666667e-01
det(A_2) = 1.444444444444444e-02
det(A_3) = 1.314814814814815e-03
det(A_4) = 3.401234567901235e-04
det(A_5) = 1.063580246913580e-04
det(A_6) = 2.222908093278465e-05
det(A_7) = 9.296890717878379e-06
det(A_8) = 4.704549611339739e-06
det(A_9) = 1.559852766346595e-06

```

Sono tutti positivi, quindi è verificata anche la condizione necessaria e sufficiente. Questo conferma a posteriori ciò che la teoria già prevedeva dalla proprietà SPD.

2.3 Calcolo della fattorizzazione LU con lu

Per calcolare la fattorizzazione ho usato:

```

[L, U, P] = lu(A);
detU = det(U);

if isequal(P, eye(n))
    disp("Nessun pivoting eseguito.");
else
    disp("Pivoting eseguito.");
end

```

Il risultato è:

- nessun pivoting eseguito, perché la matrice di permutazione P coincide con l'identità;
-

$$\det(U) = 1.5599 \times 10^{-6}.$$

Poiché nella fattorizzazione senza pivoting vale $A = LU$ con L triangolare inferiore unitaria, si ha

$$\det(A) = \det(L) \det(U) = \det(U),$$

dato che $\det(L) = 1$. Quindi il valore di $\det(U)$ coincide con il determinante della matrice del sistema. Anche questo è coerente con il fatto che tutti i minori principali siano non nulli.

2.4 Risoluzione con la fattorizzazione LU

Per risolvere il sistema $Ax = b$ usando la fattorizzazione LU ho proceduto in due passi, prima con la sostituzione in avanti e poi con quella all'indietro:

```
y = fwsub(L, b);  
xc = bckwsub(U, y);  
normXc = norm(xc);
```

La norma euclidea della soluzione ottenuta è:

$$\|x_c\|_2 = 3.918021398826541 \times 10^0.$$

Dal punto di vista numerico, questa procedura equivale a sfruttare la scomposizione del problema in due sistemi triangolari, che sono ben posti e risolvibili in modo stabile una volta costruita la LU.

2.5 Risoluzione con l'operatore \

Con il comando built-in di MATLAB:

```
xm = A \ b;  
normXm = norm(xm);
```

si ottiene:

$$\|x_m\|_2 = 3.918021398826541 \times 10^0.$$

Le due norme coincidono, quindi i due metodi producono la stessa soluzione numerica. Questo era atteso: l'operatore `\` di MATLAB applica un solutore diretto numericamente stabile, e nel nostro caso la struttura della matrice non richiede pivoting.

2.6 Confronto tra le due soluzioni

Infine ho calcolato:

```
InfNorm = norm(xm-xc, inf);
```

ottenendo un valore nullo a livello numerico:

$$\|x_m - x_c\|_\infty = 0.$$

Interpretativamente, questo significa che i due approcci coincidono entro l'aritmetica finita usata nei calcoli. In termini di analisi numerica, non emergono fenomeni di instabilità: la struttura SPD della matrice rende il problema particolarmente favorevole ai metodi diretti.

3 Analisi circuitale con metodi iterativi

3.1 Verifica che la matrice è simmetrica definita positiva

La prima verifica richiesta è:

```
if( A == A')
    if(all(eig(A)>0))
        disp("La matrice e' simmetrica e definita positiva")
    else
        disp("La matrice non e' SDP")
    end
end
```

La matrice risulta effettivamente simmetrica e definita positiva. Questo è il motivo per cui i metodi iterativi basati su tale ipotesi, in particolare gradiente, gradiente preconditionato e gradiente coniugato, sono applicabili in modo naturale.

Più precisamente:

- per Jacobi la proprietà decisiva è $\rho(B_J) < 1$;
- per gradiente e gradiente coniugato serve che A sia SPD, perché questi metodi nascono come minimizzazione del funzionale quadratico

$$\phi(x) = \frac{1}{2}x^T Ax - b^T x;$$

- il preconditionamento diagonale conserva questa struttura favorevole e migliora il condizionamento del sistema equivalente.

3.2 Metodo di Jacobi

Ho posto

$$x_0 = 0, \quad \text{toll} = 10^{-11}, \quad \text{nmax} = 1000.$$

Nel codice:

```
x0 = zeros(9,1);
toll = 1e-11;
nmax = 1000;
[xJ,kJ] = jacobi(A,b,x0,toll,nmax);
```

Il numero di iterazioni richiesto per raggiungere la tolleranza è:

$$k_J = 160.$$

L'errore relativo è stato poi calcolato con

```
true_rel_err_j = norm(xm-xJ) / norm(xm);
```

e vale:

$$\frac{\|x_m - x_J\|_2}{\|x_m\|_2} = 9.7672 \times 10^{-12}.$$

Questo valore è compatibile con la tolleranza imposta sul residuo relativo. In generale errore e residuo non coincidono, ma sono legati dal condizionamento della matrice; per questo una tolleranza molto piccola sul residuo produce qui anche un errore relativo molto piccolo sulla soluzione.

3.3 Raggio spettrale della matrice di iterazione di Jacobi

Per il metodo di Jacobi ho costruito la matrice di iterazione B_J partendo dalla scomposizione di A :

```
D = diag(diag(A));  
E = tril(A, -1);  
F = triu(A, 1);  
BJ = (eye(n) - D\A);  
rhoJ = max(abs(eig(BJ)));
```

Il raggio spettrale ottenuto è:

$$\rho_J = 8.5550 \times 10^{-1}.$$

Dal punto di vista teorico, per Jacobi vale

$$e^{(k+1)} = B_J e^{(k)}, \quad e^{(k)} = x - x^{(k)},$$

quindi

$$e^{(k)} = B_J^k e^{(0)}.$$

La condizione di convergenza del metodo è $\rho(B_J) < 1$, e nel nostro caso è soddisfatta perché $\rho_J \approx 0.8555 < 1$. Questo basta per garantire convergenza per ogni dato iniziale.

Il confronto richiesto dal testo è tra l'errore relativo osservato e $\rho_J^{k_J}$:

$$\rho_J^{k_J} = 1.4294 \times 10^{-11}.$$

Si ottiene infatti

$$9.7672 \times 10^{-12} < 1.4294 \times 10^{-11},$$

e i due valori risultano dello stesso ordine di grandezza. Questo è esattamente ciò che la teoria suggerisce: $\rho(B_J)$ descrive la velocità asintotica di convergenza, quindi ρ_J^k non va letto come una formula esatta dell'errore in norma 2, ma come il termine che governa il decadimento asintotico di B_J^k .

3.4 Metodo del gradiente dinamico

Ho ripetuto lo stesso tipo di analisi usando la funzione `graddyn.m`:

```
[xG, kG, res_rel_gdy] = graddyn(A,b,x0,nmax,toll);  
true_relative_error_g = norm(xm - xG) / norm(xm);
```

I risultati sono:

$$k_G = 348, \quad \frac{\|x_m - x_G\|_2}{\|x_m\|_2} = 1.3889 \times 10^{-11}.$$

Il metodo del gradiente dinamico per una matrice SPD coincide con il metodo di massima discesa applicato al funzionale quadratico associato al sistema. La stima teorica classica è espressa in norma A :

$$\|e^{(k)}\|_A \leq \left(\frac{\kappa(A) - 1}{\kappa(A) + 1} \right)^k \|e^{(0)}\|_A.$$

Per questo il confronto naturale è con il fattore

$$\rho_R = \frac{\kappa(A) - 1}{\kappa(A) + 1},$$

che coincide con il raggio spettrale del Richardson stazionario con parametro ottimale

$$\alpha_{opt} = \frac{2}{\lambda_{max}(A) + \lambda_{min}(A)}.$$

Nel mio script ho quindi usato:

```
kA = cond(A);
rhoR = (kA - 1) / (kA + 1);
```

da cui segue:

$$\rho_R = 9.3199 \times 10^{-1}, \quad \rho_R^{k_G} = 2.2689 \times 10^{-11}.$$

Anche qui vale il confronto numerico

$$1.3889 \times 10^{-11} < 2.2689 \times 10^{-11}.$$

Il confronto è coerente con la teoria, ma va letto correttamente: il bound teorico è in norma A , mentre io sto misurando l'errore relativo in norma 2. Le due norme sono equivalenti in dimensione finita, quindi è ragionevole aspettarsi lo stesso ordine di grandezza, ma non un'identità numerica esatta. Il fatto che il valore osservato sia leggermente più piccolo del bound è perfettamente naturale.

3.5 Metodo del gradiente preconditionato

Come preconditionatore ho scelto la parte diagonale di A :

```
P = diag(diag(A));
[xGP, kGP, res_rel_gpd] = gradprec(A,b,P,x0,nmax,toll);
true_relative_error_gp = norm(xm - xGP) / norm(xm);
```

I risultati ottenuti sono:

$$k_{GP} = 160, \quad \frac{\|x_m - x_{GP}\|_2}{\|x_m\|_2} = 9.7670 \times 10^{-12}.$$

Teoricamente, il gradiente preconditionato applicato con $P = \text{diag}(A)$ sostituisce il sistema originale con un sistema equivalente meglio condizionato. La quantità che controlla la velocità di convergenza non è più $\kappa(A)$ ma il condizionamento della matrice preconditionata. Questo spiega perché il numero di iterazioni scende da 348 a 160: il preconditionatore diagonale cattura già una parte importante della struttura di scala della matrice.

3.6 Metodo del gradiente coniugato

Per il gradiente coniugato ho usato:

```
[xGC, kGC, res_rel_gc] = gc(A, b, x0, nmax, toll);
true_relative_error_gc = norm(xm - xGC) / norm(xm);
```

I risultati sono:

$$k_{GC} = 9, \quad \frac{\|x_m - x_{GC}\|_2}{\|x_m\|_2} = 3.2587 \times 10^{-16}.$$

Questo è il risultato più interessante dal punto di vista teorico. Per una matrice SPD di dimensione n , il gradiente coniugato in aritmetica esatta converge in al più n iterazioni. Qui $n = 9$ e infatti il metodo si arresta proprio in 9 passi. Quindi il comportamento numerico osservato non è casuale, ma è esattamente quello previsto dalla teoria del metodo.

3.7 Grafico del residuo normalizzato

Per confrontare i tre metodi ho tracciato l'andamento del residuo normalizzato in funzione delle iterazioni:

```
figure
semilogy(0:kG, res_rel_gdy, 'b-', 0:kGP, res_rel_gpd, 'g-', ...
          0:kGC, res_rel_gc, 'r-', 'LineWidth', 1.2)
grid on
xlabel('Iterazioni')
ylabel('Residuo relativo')
legend('Gradiente dinamico', 'Gradiente preconditionato', ...
      'Gradiente coniugato')
```

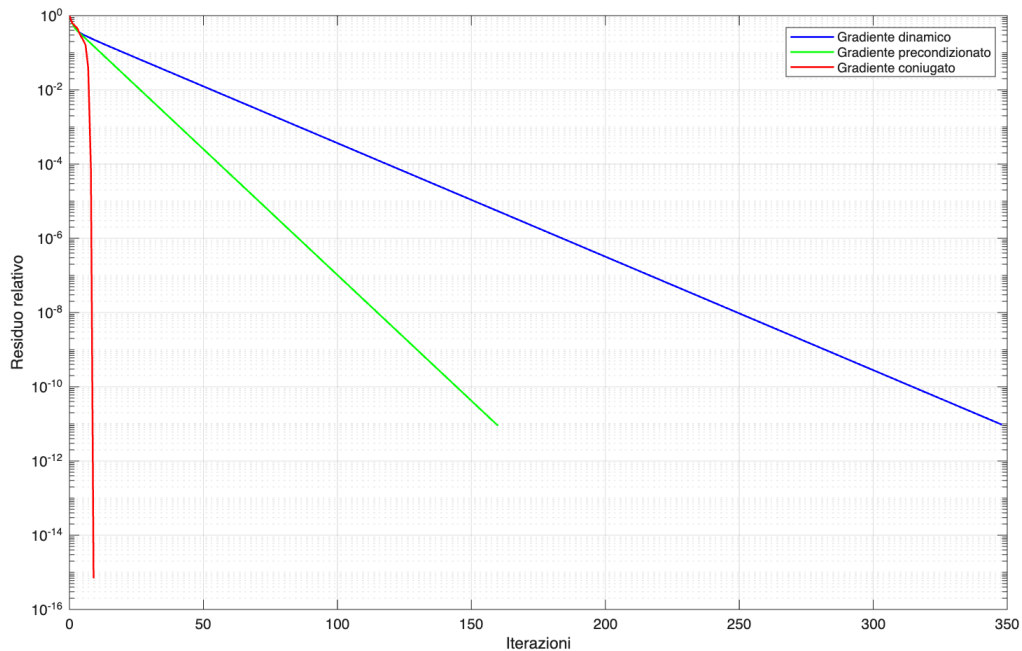


Figura 1: Andamento del residuo normalizzato per gradiente dinamico, gradiente preconditionato e gradiente coniugato.

Dal grafico si vede bene che:

- il gradiente dinamico è il più lento;
- il gradiente preconditionato converge sensibilmente più in fretta;
- il gradiente coniugato ha il decadimento più rapido del residuo.

Questo comportamento è coerente con la teoria dei tre metodi. Jacobi e gradiente dinamico hanno una velocità di convergenza governata da quantità spettrali o dal condizionamento e quindi decadono più lentamente. Il preconditionamento migliora tale quadro riducendo il numero di condizionamento effettivo. Il gradiente coniugato, invece, costruisce direzioni A -coniugate e sfrutta un meccanismo di ottimizzazione molto più efficiente, da cui il crollo rapido del residuo.

3.8 Numeri di condizionamento e commento finale

Infine ho calcolato:

```
kA = cond(A);
kA_prec = cond(P \ A);
```

ottenendo:

$$K(A) = 28.4093, \quad K(P^{-1}A) = 14.7842.$$

Il preconditionatore diagonale riduce quindi in modo evidente il numero di condizionamento. Se confronto i corrispondenti fattori teorici del metodo del gradiente ottengo:

$$\frac{K(A) - 1}{K(A) + 1} \approx 0.9320, \quad \frac{K(P^{-1}A) - 1}{K(P^{-1}A) + 1} \approx 0.8733.$$

La riduzione è significativa: un fattore più piccolo implica un decadimento più rapido dell'errore iterazione dopo iterazione. Questa è la spiegazione teorica del miglioramento osservato passando dal gradiente dinamico al gradiente preconditionato.

Nel complesso, i risultati numerici seguono la teoria:

- la proprietà SPD di A spiega sia la buona riuscita dei metodi diretti sia l'applicabilità dei metodi iterativi di tipo gradiente;
- Jacobi converge perché $\rho(B_J) < 1$, con velocità asintotica governata da ρ_J ;
- il gradiente dinamico converge con velocità legata al condizionamento di A ;
- il preconditionamento diagonale migliora il condizionamento e quindi accelera la convergenza;
- il gradiente coniugato è il più efficace e il fatto che converga in 9 iterazioni coincide con la teoria del metodo in dimensione 9.